

MEMORIA ESCRITA DEL PROYECTO

CFGS Desarrollo de Aplicaciones Web

Gestión de pacientes con bruxismo mediante aplicación web

Autor: Nombre y apellidos

Tutor: Nombre del tutor

Fecha de entrega: dd/mm/aaaa

Convocatoria: _S_____ (semestre de la convocatoria)

Documentos del proyecto: Enlace a la carpeta del Drive

Índice de contenidos

1. INTRODUCCIÓN	3
1.1. Motivación	4
1.2. Abstract	4
1.3. Objetivos propuestos (generales y específicos)	5
2. ESTADO DEL ARTE	7
3. METODOLOGÍA USADA	11
4. TECNOLOGÍAS Y HERRAMIENTAS UTILIZADAS EN EL PROYECTO	13
5. PLANIFICACIÓN, DIAGNÓSTICO Y CONTEXTO LABORAL	16
6. ANÁLISIS DEL PROYECTO	19
7. DISEÑO DEL PROYECTO	28
8. DESPLIEGUE Y PRUEBAS	34
9. CONCLUSIONES	38
10. VÍAS FUTURAS	40
11. BIBLIOGRAFÍA/WEBGRAFÍA	42
12. ANEXOS	43
12.1. Manual de instalación	43
12.2. Manual de usuario	45

En la normativa de proyectos vigente encontrarás una breve descripción de cada uno de estos apartados para saber qué información debes incluir en ellos.

1. Introducción

El presente Trabajo Final de Grado desarrolla una aplicación web para la gestión básica de pacientes con bruxismo en un contexto académico de Desarrollo de Aplicaciones Web. La propuesta se centra en una necesidad concreta: organizar la información de pacientes, citas, ejercicios recomendados y medicación asociada mediante una herramienta accesible desde navegador, con separación entre el perfil de médico y el perfil de paciente.

El tema elegido se sitúa dentro de la digitalización de procesos sanitarios de baja complejidad. No se plantea como una historia clínica profesional ni como un producto sanitario certificado, sino como una solución formativa que permite demostrar el ciclo completo de construcción de una aplicación web: análisis del problema, diseño de datos, definición de requisitos, implementación de servicios, diseño de interfaz, integración entre capas, despliegue local y validación mediante pruebas funcionales.

La actividad que desarrollaría la empresa ficticia asociada al proyecto sería la creación de soluciones web para pequeñas consultas odontológicas o clínicas que necesiten centralizar procesos administrativos y de seguimiento no invasivo. En este caso, el producto se especializa en pacientes con bruxismo porque este trastorno requiere revisiones, pautas de autocuidado y seguimiento periódico. La aplicación facilita el trabajo organizativo del profesional y ofrece al paciente un espacio para consultar la información que se le ha asignado.

El trabajo se estructura de acuerdo con la plantilla DAW_M12. En primer lugar, se explica la motivación, el resumen en inglés y los objetivos. Después se revisa el estado del arte aplicado al proyecto, se justifica la metodología y se detallan las tecnologías utilizadas. Posteriormente se desarrolla la planificación, el diagnóstico y el contexto laboral. Los apartados centrales corresponden al análisis y al diseño del sistema, donde se incluyen requisitos, casos de uso, modelo de datos, diagramas, arquitectura y decisiones técnicas. Finalmente se presentan el despliegue, las pruebas, las conclusiones, las vías futuras, la bibliografía y los anexos obligatorios.

El alcance de la aplicación ha sido delimitado para que resulte defendible y verificable. Se han incluido funcionalidades vinculadas entre sí y necesarias para una demostración coherente: registro e inicio de sesión de usuarios, diferenciación entre médico y paciente, consulta y edición de datos de perfil, creación y cancelación de citas, asignación de ejercicios, asignación de medicación y visualización de la información desde el portal del paciente.

La memoria evita tratar el bruxismo desde una perspectiva clínica exhaustiva. La información sanitaria se utiliza únicamente para justificar el caso de uso y para contextualizar el seguimiento que necesita el paciente. Las decisiones de desarrollo se

explican siempre en relación con la aplicación implementada y con las limitaciones de un proyecto académico.

El resultado final es una aplicación web funcional que integra un frontend realizado con React y Vite, un backend basado en FastAPI y una base de datos relacional MySQL. Esta estructura permite separar responsabilidades, mantener la información persistente en un modelo relacional y ofrecer interfaces diferenciadas para los usuarios. Aunque la aplicación no se orienta a producción, su diseño permite identificar con claridad qué mejoras serían necesarias para evolucionarla hacia un entorno profesional.

1.1. Motivación

La motivación principal del proyecto nace de observar que muchas tareas de gestión en pequeñas consultas todavía pueden depender de documentos aislados, hojas de cálculo, agendas manuales o comunicaciones poco estructuradas. Este modelo puede funcionar en situaciones muy reducidas, pero presenta dificultades cuando se necesita consultar rápidamente la información de un paciente, saber qué citas tiene programadas o recordar qué pautas se le han indicado.

El bruxismo permite plantear un escenario suficientemente realista sin abarcar toda la complejidad de una plataforma clínica. El paciente puede necesitar revisiones periódicas, recomendaciones de ejercicios mandibulares, pautas de autocuidado, medicación puntual o seguimiento de síntomas. Desde el punto de vista del desarrollo web, estas necesidades se traducen en entidades y flujos manejables: usuarios, pacientes, médicos, citas, catálogo de ejercicios, catálogo de medicamentos y asignaciones.

Otra razón para elegir este proyecto es que permite aplicar conocimientos del ciclo DAW de forma integrada. La aplicación requiere diseño de base de datos relacional, construcción de una API, consumo de servicios desde el frontend, formularios, validaciones, gestión de estados, pruebas manuales y documentación. No se trata de crear pantallas independientes, sino de conectar todas las partes para que la información circule entre interfaz, backend y persistencia.

Durante el desarrollo también se ha priorizado la claridad. Un Trabajo Final debe poder explicarse, defenderse y comprobarse. Por ello, se ha evitado añadir módulos no relacionados con los flujos principales. La decisión de centrar el proyecto en citas, ejercicios y medicación responde a la necesidad de construir una solución completa pero acotada, con funcionalidades que aporten valor y puedan ser probadas de extremo a extremo.

1.2. Abstract

This final project presents the development of a web application for the basic management of patients with bruxism. The system is designed for an academic context and

focuses on administrative and follow-up workflows rather than on replacing professional clinical software. It allows users to register and log in as doctors or patients, manages appointments, and supports the assignment of exercises and medication from the doctor interface.

The application follows a layered architecture composed of a React and Vite frontend, a FastAPI backend and a MySQL relational database accessed through SQLAlchemy. The frontend provides role-based views, while the backend exposes services that validate requests and persist information. The database stores the main entities of the system, including doctors, patients, appointments, exercise catalogues, medication catalogues and assignments.

The main goal of the project is to demonstrate a complete web development process: requirements analysis, database modelling, API design, interface design, implementation, local deployment and functional testing. The project also documents its limitations, especially those related to security and legal compliance in real healthcare environments. The final result is a coherent and maintainable academic application that can be extended in the future with stronger authentication, automated tests, advanced clinical history modules and production deployment.

1.3. Objetivos propuestos (generales y específicos)

Objetivo general

Desarrollar una aplicación web funcional para la gestión básica de pacientes con bruxismo, permitiendo que médicos y pacientes accedan a funcionalidades adaptadas a su rol y facilitando la organización de citas, ejercicios asignados y medicaciones asociadas.

Objetivos específicos

- Implementar el registro de usuarios diferenciando entre médico y paciente.
- Implementar el inicio de sesión y la carga de una vista inicial según el rol autenticado.
- Permitir la consulta de pacientes desde el panel del médico.
- Permitir al paciente revisar y actualizar sus datos de perfil dentro de la aplicación.
- Permitir al médico crear citas vinculadas a un paciente y a un profesional.
- Permitir la consulta de próximas citas tanto desde el perfil médico como desde el perfil paciente.
- Permitir la cancelación de citas registrando una causa de cancelación.
- Crear un catálogo de ejercicios relacionados con bruxismo y permitir su asignación a pacientes.
- Crear un catálogo de medicamentos y permitir registrar medicación asociada a pacientes.
- Mostrar en el portal del paciente las citas, ejercicios y medicaciones asignadas.

- Diseñar una base de datos relacional con más de tres tablas relacionadas mediante claves foráneas.
- Documentar pruebas funcionales de caja negra relacionadas con los casos de uso principales.

Estos objetivos específicos se han formulado como funcionalidades verificables de la aplicación. Se han evitado objetivos abstractos, de aprendizaje o no comprobables, de forma que cada uno pueda validarse mediante ejecución del sistema, revisión de la interfaz o comprobación de la base de datos.

2. Estado del Arte

El estado del arte se ha elaborado desde una perspectiva aplicada al proyecto. El objetivo no es incluir definiciones generales, sino analizar qué necesidades, soluciones y tendencias justifican la construcción de una aplicación web de apoyo para la gestión de pacientes con bruxismo. Para ello se han revisado fuentes sanitarias divulgativas y documentación oficial de tecnologías utilizadas, además de criterios de privacidad aplicables a información de salud.

El bruxismo aparece en fuentes sanitarias como una condición asociada al apretamiento o rechinar de los dientes y puede presentarse durante el sueño o en vigilia. Para este proyecto, lo relevante no es el diagnóstico clínico, sino el tipo de seguimiento que puede generar: revisiones periódicas, indicaciones de autocuidado, registro de pautas y necesidad de que el paciente recuerde ejercicios o recomendaciones. Fuentes como Mayo Clinic y NHS destacan que el seguimiento puede incluir medidas de autocuidado, consulta profesional y revisión de síntomas, lo que justifica que la aplicación organice citas y pautas sin emitir diagnósticos ni tratamientos autónomos.

En el ámbito odontológico existen plataformas comerciales que cubren agenda, historia clínica, facturación, consentimientos, comunicación con pacientes y gestión documental. Estas soluciones son completas, pero también más complejas de lo que requiere un proyecto académico. El vacío identificado para este TFG no es la inexistencia de software clínico, sino la oportunidad de construir una versión reducida, comprensible y centrada en un flujo concreto: médico, paciente, cita, ejercicio y medicación.

La tendencia general de salud digital apunta hacia soluciones que integren recursos tecnológicos, humanos y organizativos. La Organización Mundial de la Salud señala que las iniciativas de salud digital deben apoyarse en estrategias robustas y sostenibles. Trasladado al proyecto, esto implica que una aplicación sanitaria, aunque sea académica, debe diseñarse con estructura, trazabilidad de decisiones y reconocimiento de límites. Por eso el sistema se ha planteado como apoyo organizativo y no como sustituto de un acto clínico.

Una comparación con soluciones existentes muestra que las plataformas reales suelen estar orientadas a clínicas completas. Incluyen módulos de facturación, integración con laboratorios, firma de documentos o comunicación automatizada. El proyecto desarrollado no incorpora esos elementos porque desviaría el alcance y dificultaría la defensa. En cambio, se concentra en funcionalidades nucleares y comprobables: crear pacientes, registrar citas, asignar pautas y consultar información desde el portal.

Desde el punto de vista técnico, una aplicación web resulta adecuada porque permite acceso desde navegador y separa la interfaz de la lógica de negocio. Frente a una aplicación de escritorio, el modelo web reduce la dependencia del dispositivo y facilita que el

paciente use una vista sencilla sin instalar software. Frente a una aplicación móvil nativa, evita desarrollar dos versiones para distintos sistemas y se ajusta mejor a los contenidos trabajados en DAW.

La arquitectura de tres capas se alinea con la necesidad de mantener responsabilidades separadas. La interfaz no accede directamente a la base de datos; la API concentra validaciones y operaciones; y la base relacional mantiene entidades persistentes. Esta decisión también facilita las pruebas, porque cada operación funcional puede comprobarse mediante la interfaz o mediante la documentación de servicios del backend.

El uso de bases de datos relacionales sigue siendo especialmente útil cuando las entidades tienen relaciones claras. En este proyecto, las citas dependen de pacientes y médicos, las medicaciones se vinculan a pacientes y al catálogo correspondiente, y los ejercicios asignados dependen del catálogo de ejercicios. La exigencia académica de incluir una base relacional con un mínimo de tres tablas se supera ampliamente, y las claves foráneas permiten explicar la integridad del modelo.

Respecto al frontend, la elección de React responde a la necesidad de construir una interfaz dinámica por roles. La documentación oficial describe React como una biblioteca para construir interfaces a partir de componentes. En la aplicación, esa idea se refleja en pantallas diferenciadas para médico y paciente, formularios reutilizables y renderizado condicionado por el estado de sesión. Esta tecnología aparece dentro de las tecnologías consideradas válidas en el ciclo DAW.

En el backend, FastAPI se ha elegido por su integración con Python, la validación de datos y la generación automática de documentación de servicios. Aunque FastAPI no aparezca literalmente en la lista de tecnologías del ciclo, su uso queda justificado porque se apoya en Python, lenguaje incluido en el temario, y permite construir APIs de forma clara para un proyecto académico. Su documentación oficial lo presenta como un framework para crear APIs con anotaciones estándar de Python.

SQLAlchemy se ha utilizado como capa de acceso a datos porque evita repetir consultas manuales y permite representar tablas mediante modelos. Esta decisión mejora la mantenibilidad, aunque exige documentar su uso por tratarse de una herramienta específica. En el proyecto se utiliza de forma limitada y aplicada: modelos para tablas, relaciones, consultas y operaciones de persistencia.

Vite se ha incorporado como herramienta de desarrollo para React porque simplifica la ejecución local del frontend y la generación de una versión de producción. Su uso se justifica por productividad y por reducir configuración accidental. No sustituye a React ni modifica la arquitectura, sino que actúa como herramienta de construcción del cliente.

En cuanto a privacidad, cualquier proyecto que trate información de pacientes debe ser prudente. La guía de la Agencia Española de Protección de Datos para profesionales sanitarios recuerda que los datos de salud exigen especial atención. Por ello, la memoria declara de forma explícita que el sistema es académico, que no está preparado para producción y que requeriría cifrado robusto, control de accesos, trazabilidad, copias de seguridad y cumplimiento normativo antes de utilizarse en un entorno real.

La revisión crítica permite concluir que el proyecto se sitúa en un espacio intermedio: toma necesidades reales de seguimiento y organización, pero evita pretender competir con plataformas clínicas completas. Su aportación académica consiste en demostrar que un problema sanitario acotado puede convertirse en una aplicación web estructurada, con roles, persistencia y pruebas, siempre que se indiquen sus límites y se justifiquen las tecnologías no vistas explícitamente en el ciclo.

2.1. Comparativa aplicada de alternativas

Para definir el alcance se compararon tres alternativas: una plataforma clínica completa, una agenda generalista y una aplicación especializada en seguimiento básico de bruxismo. La plataforma completa ofrecía más funcionalidades, pero obligaba a tratar módulos de facturación, documentación clínica y permisos complejos. La agenda generalista resolvía parcialmente la organización temporal, pero no permitía relacionar pautas, catálogos y paciente. La alternativa seleccionada fue una aplicación especializada y reducida, porque se ajusta mejor a un TFG y permite defender una solución completa sin invadir áreas clínicas sensibles.

Esta comparación condicionó varias decisiones. El sistema no calcula diagnósticos, no recomienda medicación por sí mismo y no sustituye la valoración del profesional. En su lugar, registra información introducida por el médico y la presenta al paciente de forma organizada. Esta frontera funcional es importante porque diferencia una herramienta de apoyo de una herramienta clínica autónoma.

Tabla 1.1. Comparativa de alternativas de solución. Fuente: elaboración propia.

Alternativa analizada	Ventaja principal	Limitación para este TFG	Decisión
Plataforma clínica completa	Cubre agenda, historia clínica, facturación y documentos.	Excede el alcance y requiere garantías legales y técnicas avanzadas.	Descartada como alcance principal.
Agenda web generalista	Permite organizar citas con menor complejidad.	No resuelve asignación de ejercicios ni medicación asociada al paciente.	Insuficiente para el problema elegido.
Aplicación especializada en bruxismo	Conecta paciente, cita y pautas de seguimiento básico.	No cubre todas las necesidades de una consulta real.	Seleccionada por equilibrio entre alcance y valor académico.

2.2. Tendencias y vacíos detectados

La tendencia observada en herramientas sanitarias es avanzar hacia mayor digitalización, centralización de datos y acceso remoto. Sin embargo, muchas soluciones reales se orientan a organizaciones completas y no a proyectos pequeños de aprendizaje. El vacío que aprovecha este proyecto es crear una herramienta con alcance limitado, pero suficientemente estructurada como para mostrar arquitectura, roles, persistencia, pruebas y documentación.

Otro vacío detectado es la distancia entre una simple agenda y un sistema clínico. En el caso del bruxismo, no basta con registrar citas si el objetivo es que el paciente recuerde pautas asignadas; pero tampoco es necesario construir una historia clínica completa para demostrar la solución. La aplicación se sitúa en ese punto medio: registra datos operativos, no diagnósticos complejos.

Finalmente, la revisión de privacidad muestra que cualquier evolución debe evitar improvisar con datos sensibles. Esta conclusión ha influido en la memoria, que declara límites de seguridad y propone mejoras futuras antes de un uso real. De esta forma, el estado del arte no solo justifica la idea, sino que también delimita qué no debe hacer el proyecto en su versión actual.

Tabla 1. Síntesis crítica del estado del arte aplicado al proyecto. Fuente: elaboración propia.

Aspecto revisado	Aplicación al proyecto	Decisión adoptada
Seguimiento del bruxismo	Necesidad de citas, ejercicios y pautas de consulta para el paciente.	Se incorporan citas, ejercicios asignados y medicación asociada.
Software clínico existente	Soluciones comerciales son completas pero exceden el alcance académico.	Se desarrolla una versión reducida y defendible.
Salud digital	Necesidad de soluciones sostenibles y bien estructuradas.	Se delimita el sistema como apoyo organizativo.
Privacidad sanitaria	Los datos de salud exigen medidas avanzadas.	Se documentan límites y mejoras futuras de seguridad.
Tecnologías web	Interfaz, API y base de datos pueden separarse por capas.	Se adopta frontend React, backend FastAPI y MySQL.

3. Metodología usada

La metodología empleada ha sido incremental e iterativa. Esta decisión se ajusta al tamaño del proyecto y a la necesidad de comprobar cada módulo antes de continuar con el siguiente. En lugar de realizar primero todo el análisis y dejar la validación para el final, el trabajo se organizó en ciclos de avance: definición de requisitos, diseño mínimo, implementación, prueba y revisión.

El ciclo de vida aplicado combina planificación inicial con revisiones sucesivas. Primero se delimitó el alcance: una aplicación web para médico y paciente, con gestión de citas, ejercicios y medicación. Después se diseñó el modelo de datos necesario para soportar esas funcionalidades. A partir de ahí se desarrolló el backend, se conectó el frontend y se realizaron pruebas manuales. Cada fase produjo ajustes en la siguiente, especialmente en nombres de campos, validaciones y flujos de usuario.

La metodología incremental fue adecuada porque el sistema contiene módulos relativamente independientes pero conectados. Por ejemplo, antes de implementar la asignación de ejercicios era necesario disponer de pacientes registrados y de un catálogo de ejercicios. Antes de probar el portal del paciente era necesario que existieran citas y asignaciones. Esta dependencia entre módulos obligó a priorizar una secuencia lógica de desarrollo.

3.1. Ciclo de vida aplicado

El ciclo de vida se organizó en ocho fases. Cada fase tuvo un resultado verificable y se conectó con la siguiente. Esta organización permitió controlar el avance del proyecto y detectar incoherencias antes de la entrega final.

Tabla 2. Fases del ciclo de vida aplicado al proyecto. Fuente: elaboración propia.

Fase	Objetivo aplicado	Resultado obtenido
1. Análisis inicial	Determinar usuarios, problema y funcionalidades mínimas.	Roles médico y paciente, alcance funcional y flujo general.
2. Diseño de datos	Construir un modelo relacional coherente.	Tablas principales, claves primarias, claves foráneas y catálogos.
3. Backend inicial	Crear servicios de registro, login y consulta.	API base con validación y conexión a MySQL.
4. Gestión de citas	Permitir crear, consultar y cancelar citas.	Operaciones funcionales para agenda y portal.
5. Ejercicios y medicación	Añadir catálogos y asignaciones.	Pautas visibles desde el panel del paciente.
6. Frontend por roles	Separar la experiencia de médico y paciente.	Pantallas y formularios conectados a la API.
7. Pruebas manuales	Verificar casos de uso mediante caja negra.	Tabla de pruebas con entradas, resultados esperados y resultado real.
8. Documentación	Alinear memoria, aplicación y defensa.	Memoria estructurada, anexos y criterios de entrega.

3.2. Organización de iteraciones

La primera iteración se dedicó a validar la idea base. Se identificó que el proyecto debía incluir al menos dos roles y que el médico tenía que disponer de herramientas de gestión, mientras que el paciente debía tener una vista de consulta. Esta iteración permitió descartar funcionalidades demasiado amplias, como facturación o historial clínico completo.

La segunda iteración se centró en la base de datos y en el backend. Se comprobaron las relaciones entre pacientes, médicos y citas, y se añadieron catálogos para evitar que cada asignación fuese texto libre. Esta decisión facilitó la coherencia del sistema y permitió que las pautas asignadas tuvieran una fuente común.

La tercera iteración abordó el frontend. Se construyeron vistas diferenciadas para cada rol y se centralizaron las llamadas a la API. En esta fase se ajustaron formularios y mensajes para que el usuario entendiera qué datos eran obligatorios y qué acciones podía realizar.

La cuarta iteración fue de integración y limpieza. Se revisó que las rutas activas correspondieran con funcionalidades visibles, se eliminaron elementos que no aportaban valor a la defensa y se prepararon pruebas de caja negra sobre los flujos principales. Esta fase fue clave para que la memoria describiera el sistema real y no una versión idealizada.

3.3. Gestión de cambios

Durante el proyecto se tomaron decisiones de reducción de alcance. La más importante fue no incluir módulos de historia clínica avanzada ni informes estadísticos, ya que requerían más tiempo y podían introducir datos sanitarios que el sistema académico no está preparado para tratar. También se evitó incluir código no conectado con la interfaz, porque dificultaría la evaluación y podría generar inconsistencias entre memoria y aplicación.

La metodología ha permitido mantener una relación directa entre objetivos, funcionalidades, pruebas y documentación. Cada requisito funcional tiene un caso de uso asociado y, en la medida de lo posible, una prueba funcional de caja negra. Esta trazabilidad ayuda a defender el proyecto y a justificar qué partes se han implementado realmente.

4. Tecnologías y herramientas utilizadas en el proyecto

Este apartado detalla las tecnologías y herramientas utilizadas, explicando su papel dentro del proyecto y la causa de su elección. Se diferencia entre tecnologías vistas o alineadas con el ciclo DAW y herramientas complementarias que, aunque no aparezcan literalmente en el listado, se han justificado por su utilidad directa en la aplicación.

La base tecnológica se apoya en elementos válidos del ciclo: Python, JavaScript, HTML, CSS, JSON, SQL, React y MySQL. Sobre esa base se han incorporado FastAPI, SQLAlchemy, Pydantic, Vite y Uvicorn. Estas herramientas no se han elegido para aumentar artificialmente la complejidad, sino para organizar el desarrollo y hacer posible una API clara, validada y conectada con la base de datos.

Tabla 3. Tecnologías y herramientas utilizadas. Fuente: elaboración propia.

Tecnología/Herramienta	Uso en el proyecto	Justificación
Python	Lenguaje del backend.	Lenguaje incluido en el ciclo y adecuado para construir servicios con sintaxis clara.
FastAPI	Framework de la API REST.	Se justifica por la generación de documentación de servicios, validación y rapidez de desarrollo sobre Python.
SQLAlchemy	Capa ORM de acceso a datos.	Evita duplicar consultas y mantiene modelos vinculados a tablas relacionales.
Pydantic	Validación de esquemas de entrada y salida.	Ayuda a controlar datos recibidos por la API y reduce errores en formularios.
MySQL	Sistema gestor de base de datos relacional.	Tecnología válida del ciclo; permite claves foráneas y relaciones entre más de tres tablas.
React	Biblioteca de interfaz.	Tecnología válida; facilita vistas por rol y componentes reutilizables.
Vite	Herramienta de desarrollo del frontend.	Se justifica porque simplifica ejecución y construcción de la aplicación React.
JavaScript, HTML y CSS	Lógica de cliente, estructura y estilos.	Tecnologías base del desarrollo web vistas en el ciclo.
Uvicorn	Servidor local del backend.	Necesario para ejecutar FastAPI en entorno de desarrollo.
MySQL Workbench o cliente equivalente	Preparación y revisión de la base de datos.	Permite ejecutar scripts, revisar tablas y comprobar relaciones.
Git	Control y limpieza del proyecto.	Permite mantener versiones y excluir dependencias generadas.

Python se ha utilizado en el backend porque permite construir una aplicación legible y modular. En este proyecto su uso se concentra en la definición de modelos, servicios y rutas. La elección se alinea con el listado de tecnologías válidas del ciclo y facilita explicar el comportamiento del servidor durante la defensa.

FastAPI se utiliza como framework de la API. Su valor principal en el proyecto es que permite agrupar operaciones por funcionalidad, validar datos y disponer de documentación interactiva de servicios. Aunque no figure literalmente en la tabla de tecnologías vistas, se apoya en Python y cumple una función clara: exponer la lógica del sistema al frontend de forma ordenada.

SQLAlchemy se emplea para conectar el backend con MySQL mediante modelos. La alternativa habría sido escribir consultas SQL manuales en cada operación. Se eligió SQLAlchemy porque reduce repetición, centraliza la representación de tablas y ayuda a mantener la relación entre código y base de datos. Su uso queda documentado porque es una herramienta adicional respecto a los contenidos principales del ciclo.

MySQL se ha elegido como SGBD relacional porque cumple el requisito de base de datos relacional y permite modelar entidades con claves foráneas. La aplicación contiene más de tres tablas relacionadas, entre ellas pacientes, médicos, citas, medicaciones, catálogos y ejercicios asignados. Esta decisión facilita demostrar integridad referencial y consultas estructuradas.

React se utiliza para la interfaz porque permite mostrar pantallas dinámicas según el rol del usuario. La aplicación no se limita a páginas estáticas: necesita formularios, mensajes de validación, listados y cambios de vista. React permite organizar esos elementos en componentes y manejar el estado de sesión de forma más clara que con JavaScript sin estructura.

Vite actúa como herramienta de desarrollo y construcción del frontend. Se incorporó para simplificar la configuración, acelerar el servidor de desarrollo y generar una versión preparada para revisión. No introduce una dependencia funcional para el usuario final, sino una mejora en el flujo de trabajo del desarrollador.

Pydantic se usa para validar la información que entra y sale por la API. En una aplicación con formularios es importante controlar que los campos obligatorios existan y que los datos tengan el formato esperado. Esta herramienta se justifica porque complementa a FastAPI y reduce errores de integración.

Las herramientas auxiliares, como Git y un cliente de MySQL, se han usado para mantener el proyecto ordenado. Aunque no formen parte del producto final, son necesarias para preparar la entrega, comprobar el estado de la base de datos y evitar incluir archivos generados como dependencias instaladas o cachés.

4.1. Relación con tecnologías del ciclo DAW

Las tecnologías nucleares del proyecto están vinculadas al listado de tecnologías consideradas válidas: Python, JavaScript, HTML, CSS, JSON, SQL, React y MySQL. Las

herramientas complementarias se justifican porque resuelven necesidades concretas del sistema y no sustituyen la base del aprendizaje del ciclo.

Tabla 4. Encaje de tecnologías con el ciclo DAW. Fuente: elaboración propia.

Elemento	Aparece o se relaciona con el ciclo	Motivo de validez en el proyecto
Python	Sí	Lenguaje principal del backend.
SQL y MySQL	Sí	Persistencia relacional obligatoria.
React	Sí	Interfaz web por componentes.
HTML, CSS y JavaScript	Sí	Estructura, estilos y comportamiento del frontend.
FastAPI	Tecnología similar/complementaria	Framework de API sobre Python; se documenta y justifica.
SQLAlchemy	Herramienta complementaria	ORM para acceso a datos relacionales; se documenta y justifica.
Vite	Herramienta complementaria	Herramienta de construcción para React; se documenta y justifica.

5. Planificación, Diagnóstico y Contexto Laboral

La planificación del proyecto se ha realizado considerando una duración académica limitada y un alcance funcional controlado. El objetivo no era construir una plataforma sanitaria completa, sino desarrollar una aplicación web que pudiera entregarse, ejecutarse y defenderse. Por ello, la planificación combina tareas de análisis, implementación, pruebas y documentación.

El diagnóstico inicial identificó una necesidad concreta: centralizar información básica de pacientes con bruxismo y facilitar que médico y paciente consulten datos relacionados con citas y pautas. A nivel laboral, el proyecto podría situarse en una pequeña empresa de desarrollo que ofrece soluciones web personalizadas a consultas odontológicas. La aplicación se plantea como herramienta interna de apoyo, no como producto sanitario certificado.

5.1. Diagnóstico del problema

- La información de pacientes puede quedar dispersa entre notas, documentos y agendas no conectadas.
- El profesional necesita consultar próximas citas y pautas asignadas sin revisar varias fuentes.
- El paciente puede olvidar ejercicios o indicaciones si no dispone de un espacio centralizado.
- Las asignaciones de ejercicios y medicación deben vincularse a un paciente concreto para evitar confusiones.
- La aplicación debe evitar un alcance clínico excesivo y centrarse en la gestión básica y demostrable.

A partir de este diagnóstico se priorizaron módulos que resuelven necesidades directas. La agenda atiende el problema de organización temporal; el portal del paciente atiende la consulta de pautas; los catálogos reducen la introducción de textos libres; y la base relacional garantiza que las asignaciones dependan de pacientes existentes.

5.2. Diagrama de Gantt: estimación y tiempo real

Tabla 5. Planificación tipo Gantt con estimación y tiempo real. Fuente: elaboración propia.

Tarea	Estimación	Tiempo real	Desviación / comentario
Definición de alcance y objetivos	Semana 1	Semana 1	Sin desviación; el alcance se mantuvo centrado.
Análisis de requisitos	Semana 1-2	Semana 1-2	Se añadieron requisitos de cancelación y portal.
Diseño de base de datos	Semana 2	Semana 2-3	Ligera ampliación por catálogos y asignaciones.

Tarea	Estimación	Tiempo real	Desviación / comentario
Backend de autenticación de pacientes y	Semana 3	Semana 3-4	Ajustes en validaciones de perfiles.
Backend de citas	Semana 4	Semana 4-5	Se añadió control de cita duplicada y motivo de cancelación.
Backend de ejercicios y medicación	Semana 5	Semana 5-6	Se completó con catálogos y asignaciones.
Frontend de login y registro	Semana 5	Semana 6	Dependía de la estabilidad de la API.
Frontend de panel médico	Semana 6	Semana 6-7	Se integraron listados y acciones de asignación.
Frontend de portal paciente	Semana 7	Semana 7-8	Se ajustó la presentación de citas y pautas.
Pruebas manuales	Semana 8	Semana 8-9	Se repitieron pruebas tras correcciones.
Limpieza y documentación	Semana 9	Semana 9-10	Se eliminaron elementos no conectados y se preparó memoria.

La desviación más significativa se produjo en la integración frontend-backend. Algunas operaciones parecían resueltas de forma aislada, pero al conectarlas con formularios reales se detectaron campos obligatorios, estados y mensajes que debían ajustarse. Esta desviación es habitual en proyectos web porque la integración revela problemas que no aparecen durante el diseño inicial.

5.3. Análisis DAFO

Tabla 6. DAFO interno del proyecto. Fuente: elaboración propia.

Fortalezas	Debilidades
Arquitectura separada por capas; funcionalidades conectadas; base de datos relacional con relaciones claras; proyecto defendible y ejecutable en local.	Seguridad básica; ausencia de pruebas automatizadas; falta de despliegue remoto; alcance clínico limitado; dependencia de configuración local.

Tabla 7. DAFO externo del proyecto. Fuente: elaboración propia.

Oportunidades	Amenazas
Crecimiento de la digitalización en consultas pequeñas; posibilidad de añadir recordatorios, informes y mejor experiencia de usuario; evolución hacia despliegue cloud.	Exigencias legales para datos de salud; competencia de software clínico profesional; necesidad de mantenimiento; riesgo de tratar datos sensibles sin medidas avanzadas.

El DAFO muestra que la principal fortaleza del proyecto es la coherencia entre módulos. La aplicación no presenta funcionalidades aisladas, sino un flujo que conecta usuario, cita, paciente, ejercicio y medicación. Esta conexión facilita la defensa porque permite demostrar operaciones completas desde la interfaz hasta la base de datos.

La principal debilidad es la seguridad. El sistema se ha construido para un entorno académico local y no debe utilizarse con datos reales de salud. Para una evolución profesional sería imprescindible implantar autenticación robusta, cifrado de contraseñas, control de permisos, registro de auditoría, copias de seguridad y cumplimiento normativo.

La oportunidad más clara es ampliar el sistema hacia herramientas de seguimiento preventivo y comunicación con el paciente. Sin embargo, esa oportunidad también genera amenazas porque el tratamiento de datos sanitarios tiene una carga legal y ética importante. Por ello, cualquier crecimiento futuro debe ir acompañado de seguridad, documentación y limitación clara de responsabilidades.

5.4. Contexto laboral

En un contexto laboral, la aplicación podría ser desarrollada por una pequeña empresa de software a medida para consultas odontológicas. El cliente no solicitaría una plataforma clínica integral, sino una herramienta de apoyo para organizar pacientes y pautas. El equipo mínimo podría incluir un desarrollador web, un responsable de análisis funcional y un profesional sanitario que validara el vocabulario y el flujo de trabajo.

El producto tendría que evolucionar de forma gradual. La primera versión, equivalente a este TFG, permitiría demostrar la viabilidad técnica. Una segunda versión debería abordar seguridad, despliegue, copias de seguridad y normativa. Una versión profesional requeriría revisión legal y validación del uso sanitario, especialmente si se almacenan datos reales o se envían recordatorios con información sensible.

6. Análisis del proyecto

El análisis es el apartado central de la memoria porque define las necesidades que debe cubrir la aplicación y la forma en que se representan dentro del sistema. En este proyecto, el análisis se ha orientado a funcionalidades verificables y a una estructura de datos relacional. Se incluyen requisitos funcionales, requisitos no funcionales, actores, casos de uso, especificaciones, modelo entidad-relación y diagrama de clases.

La finalidad del sistema es proporcionar una herramienta de gestión básica para pacientes con bruxismo, sin sustituir la intervención profesional. El médico administra citas y asignaciones; el paciente consulta su información y puede revisar sus próximas citas y pautas. Esta separación de responsabilidades condiciona todos los requisitos y decisiones posteriores.

6.1. Descripción general del sistema

El sistema se ejecuta como una aplicación web con tres componentes: cliente, servidor y base de datos. El cliente muestra la interfaz de usuario; el servidor recibe y valida las operaciones; la base de datos guarda los registros. La aplicación distingue entre dos roles principales: médico y paciente. El rol determina qué opciones se muestran y qué operaciones resultan relevantes.

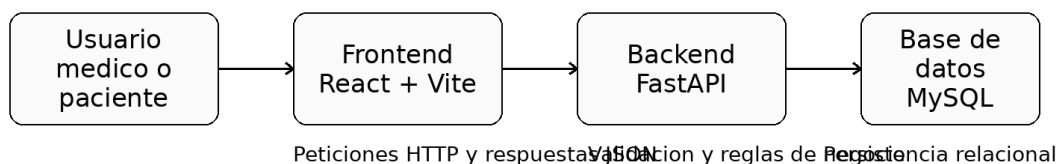


Figura 1. Arquitectura lógica de la aplicación. Fuente: elaboración propia.

6.2. Actores del sistema

Tabla 8. Actores del sistema. Fuente: elaboración propia.

Actor	Descripción	Responsabilidades principales
Médico	Usuario profesional que gestiona pacientes, citas y pautas.	Consultar pacientes, crear citas, cancelar citas, asignar ejercicios y medicación.
Paciente	Usuario que consulta su información y sus pautas.	Iniciar sesión, consultar citas, editar datos de perfil y revisar asignaciones.
Sistema	Conjunto de frontend, backend y base de datos.	Validar operaciones, persistir datos, devolver respuestas y mantener relaciones.

6.3. Requisitos funcionales

Tabla 9. Requisitos funcionales. Fuente: elaboración propia.

Código	Nombre	Descripción
RF01	Registro de paciente	El sistema permite crear una cuenta de paciente con los datos obligatorios definidos.
RF02	Registro de médico	El sistema permite crear una cuenta de médico con datos profesionales mínimos.
RF03	Inicio de sesión	El sistema valida credenciales y devuelve el rol para cargar la vista correspondiente.
RF04	Consulta de pacientes	El médico puede visualizar el listado de pacientes registrados.
RF05	Edición de perfil de paciente	El paciente puede actualizar información básica de su perfil.
RF06	Creación de cita	El médico puede crear una cita asociada a paciente, médico, fecha y tipo.
RF07	Consulta de próximas citas	Médico y paciente pueden consultar citas futuras relacionadas con su perfil.
RF08	Cancelación de cita	El sistema permite cancelar una cita registrando una causa obligatoria.
RF09	Consulta de catálogo de ejercicios	El médico puede ver ejercicios disponibles para asignar.
RF10	Asignación de ejercicio	El médico puede asignar un ejercicio concreto a un paciente.
RF11	Consulta de catálogo de medicamentos	El médico puede consultar medicamentos disponibles.
RF12	Asignación de medicación	El médico puede registrar medicación asociada a un paciente.
RF13	Portal del paciente	El paciente puede consultar sus citas, ejercicios y medicaciones.
RF14	Resumen de paciente	El sistema puede mostrar una visión agrupada de datos y pautas asociadas.

6.4. Requisitos no funcionales

Tabla 10. Requisitos no funcionales. Fuente: elaboración propia.

Código	Nombre	Descripción
RNF01	Usabilidad	La interfaz debe ser comprensible para una demostración académica y diferenciar acciones por rol.
RNF02	Mantenibilidad	El proyecto debe separar frontend, backend y base de datos.
RNF03	Integridad	La base de datos debe evitar asignaciones a pacientes inexistentes mediante relaciones.
RNF04	Portabilidad local	El sistema debe poder ejecutarse en un entorno local

Código	Nombre	Descripción
		con dependencias documentadas.
RNF05	Seguridad académica	La aplicación debe declarar sus límites y no presentarse como sistema de producción.
RNF06	Trazabilidad	Las funcionalidades descritas deben tener relación con casos de uso y pruebas.
RNF07	Claridad de entrega	El proyecto no debe incluir módulos desconectados que dificulten la corrección.

6.5. Casos de uso

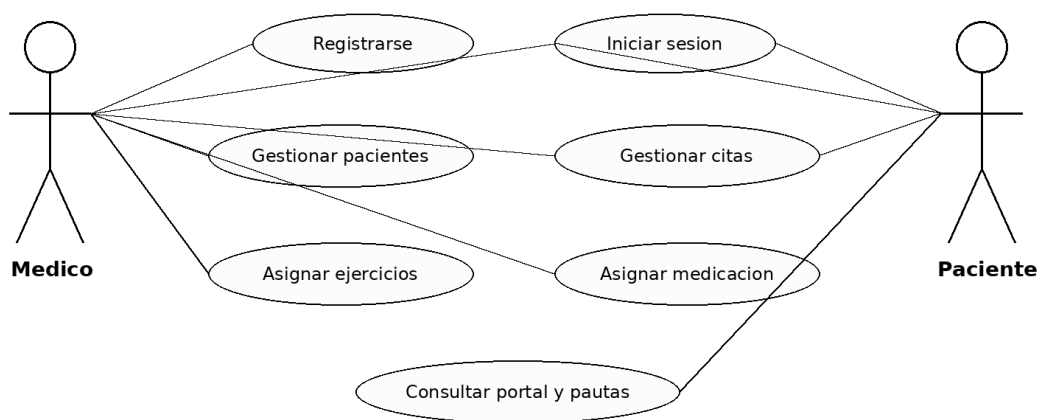


Figura 2. Diagrama de casos de uso principales. Fuente: elaboración propia.

Los casos de uso se han definido a partir de las acciones que realmente realiza la aplicación. No se incluyen procesos clínicos avanzados ni operaciones no implementadas. La siguiente tabla resume los casos y su relación con los requisitos funcionales.

Tabla 11. Relación entre casos de uso y requisitos. Fuente: elaboración propia.

Caso de uso	Actor principal	Requisitos relacionados
CU01. Registrarse	Médico o paciente	RF01, RF02
CU02. Iniciar sesión	Médico o paciente	RF03
CU03. Consultar pacientes	Médico	RF04
CU04. Editar perfil	Paciente	RF05
CU05. Crear cita	Médico	RF06
CU06. Consultar citas	Médico o paciente	RF07
CU07. Cancelar cita	Médico o paciente	RF08
CU08. Asignar ejercicio	Médico	RF09, RF10
CU09. Asignar medicación	Médico	RF11, RF12
CU10. Consultar portal	Paciente	RF13, RF14

6.6. Especificaciones de casos de uso

Tabla 12. Especificaciones resumidas de casos de uso. Fuente: elaboración propia.

Caso	Precondición	Flujo principal	Resultado
CU01. Registrarse	Usuario no registrado	El usuario selecciona rol, completa datos obligatorios y confirma.	Cuenta creada si el correo no existe; error si faltan datos o el correo está duplicado.
CU02. Iniciar sesión	Usuario registrado	El usuario introduce email y contraseña y solicita acceso.	Se carga el panel de médico o paciente; si falla, se muestra error.
CU03. Consultar pacientes	Médico autenticado	El médico accede al listado de pacientes.	Se muestran pacientes registrados para selección y gestión.
CU04. Editar perfil	Paciente autenticado	El paciente modifica campos permitidos y guarda cambios.	El sistema actualiza los datos y muestra confirmación.
CU05. Crear cita	Médico autenticado y paciente existente	El médico selecciona paciente, fecha, tipo y profesional.	La cita queda registrada si no hay conflicto básico.
CU06. Consultar citas	Usuario autenticado	El usuario entra en la sección de agenda o citas.	Se muestran citas futuras relacionadas con su perfil.
CU07. Cancelar cita	Usuario autenticado con cita asociada	El usuario solicita cancelar e introduce motivo.	La cita cambia a estado cancelado y se guarda la causa.
CU08. Asignar ejercicio	Médico autenticado y paciente existente	El médico elige un ejercicio y registra frecuencia e inicio.	El ejercicio aparece en el portal del paciente.
CU09. Asignar medicación	Médico autenticado y paciente existente	El médico elige medicamento y registra dosis, frecuencia y motivo.	La medicación aparece asociada al paciente.
CU10. Consultar portal	Paciente autenticado	El paciente accede a su panel y revisa secciones.	Se muestran datos, citas, ejercicios y medicaciones.

6.7. Modelo Entidad-Relación

La base de datos se ha diseñado como un modelo relacional con más de tres tablas relacionadas. Las entidades principales son pacientes, médicos, citas, catálogos y asignaciones. Las relaciones garantizan que una cita o pauta no exista de forma aislada, sino vinculada a un paciente y, cuando corresponde, a un profesional o catálogo.

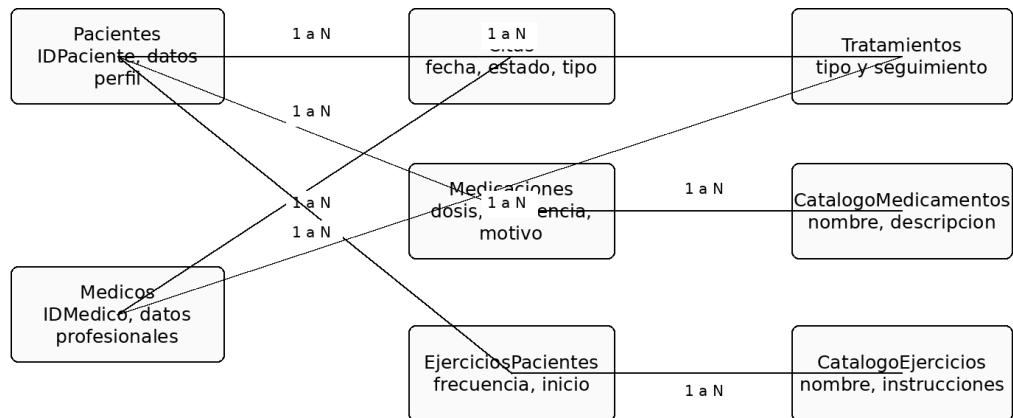


Figura 3. Diagrama Entidad-Relación simplificado del proyecto. Fuente: elaboración propia.

Tabla 13. Descripción de entidades principales. Fuente: elaboración propia.

Entidad	Propósito en el sistema	Relaciones destacadas
Pacientes	Almacena el perfil del paciente y permite asociar citas, medicaciones y ejercicios.	Tiene muchas citas, medicaciones y ejercicios asignados.
Médicos	Representa al profesional que gestiona agenda y pautas.	Puede atender muchas citas y prescribir tratamientos.
Citas	Registra fecha, estado, tipo y posible motivo de cancelación.	Pertenece a un paciente y se asocia a un médico.
Catálogo de ejercicios	Define ejercicios disponibles para asignar.	Un ejercicio del catálogo puede asignarse a muchos pacientes.
Ejercicios de pacientes	Registra la asignación concreta de un ejercicio a un paciente.	Relaciona paciente y ejercicio del catálogo.
Catálogo de medicamentos	Define medicamentos disponibles para seleccionar.	Un medicamento puede aparecer en varias asignaciones.
Medicaciones	Registra dosis, frecuencia, motivo y fecha de inicio.	Pertenece a un paciente y puede referenciar catálogo.
Tratamientos	Permite clasificar intervenciones o seguimiento si se amplía el sistema.	Relaciona pacientes, médicos y tipos de tratamiento.

6.8. Reglas de negocio y validaciones

- No debe existir más de una cuenta con el mismo correo electrónico dentro del flujo de registro.
- El registro de paciente debe incluir los campos mínimos necesarios para identificar su perfil en la aplicación.
- El registro de médico debe incluir información profesional mínima para distinguirlo del perfil paciente.
- Una cita debe estar asociada a un paciente existente y a un profesional seleccionado.
- Una cita cancelada debe conservar un motivo de cancelación para que la acción sea trazable.
- Una asignación de ejercicio debe referenciar un paciente y un ejercicio del catálogo.

- Una medicación asignada debe quedar vinculada a un paciente y contener dosis, frecuencia y motivo.
- El portal del paciente solo debe mostrar la información asociada a ese paciente autenticado.

Estas reglas han sido seleccionadas porque afectan directamente a la coherencia del sistema. No se han incluido reglas clínicas avanzadas, como cálculo de dosis o diagnóstico, porque exceden el alcance del proyecto y requerirían validación profesional.

6.9. Diagrama de clases

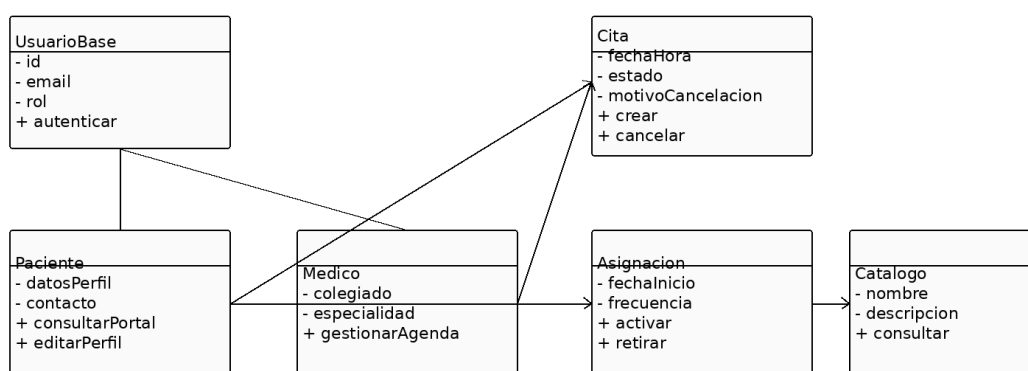


Figura 4. Diagrama de clases conceptual del sistema. Fuente: elaboración propia.

El diagrama de clases representa una visión conceptual. No reproduce código fuente, sino responsabilidades del dominio: usuarios, pacientes, médicos, citas, catálogos y asignaciones. Esta visión facilita explicar cómo las entidades se transforman en modelos de backend y en vistas dentro del frontend.

6.10. Flujos de información

El análisis también contempla cómo viaja la información entre usuarios y capas. El médico introduce o consulta datos desde la interfaz; el frontend envía la petición al backend; el backend valida la operación; y la base de datos almacena o recupera la información. El paciente consume una parte de esos datos desde su portal, principalmente citas y pautas asignadas.

Tabla 14. Flujos de información principales. Fuente: elaboración propia.

Flujo	Origen	Destino	Información implicada
Registro	Formulario de usuario	Tablas de pacientes o médicos	Datos de perfil y credenciales de prueba.
Creación de cita	Panel médico	Tabla de citas	Paciente, profesional, fecha, tipo y estado.
Cancelación de cita	Panel médico o paciente	Tabla de citas	Estado cancelado, motivo y fecha de

Flujo	Origen	Destino	Información implicada
			cancelación.
Asignación de ejercicio	Panel médico	Tabla de ejercicios de pacientes	Paciente, ejercicio, frecuencia e inicio.
Asignación de medicación	Panel médico	Tabla de medicaciones	Paciente, medicamento, dosis, frecuencia y motivo.
Consulta de portal	Panel paciente	API y base de datos	Citas futuras, ejercicios y medicaciones asociadas.

6.11. Criterios de aceptación

Los criterios de aceptación permiten comprobar cuándo un requisito puede considerarse completado. Se han redactado en forma observable para que puedan validarse durante la defensa sin revisar el código fuente.

Tabla 15. Criterios de aceptación de requisitos. Fuente: elaboración propia.

Requisito	Criterio de aceptación
RF01-RF03	Un usuario puede registrarse e iniciar sesión y el sistema muestra el panel correspondiente a su rol.
RF04-RF05	El médico puede consultar pacientes y el paciente puede actualizar datos permitidos de su perfil.
RF06-RF08	Una cita puede crearse, listarse como próxima y cancelarse únicamente si se introduce motivo.
RF09-RF10	Un ejercicio seleccionado del catálogo queda visible en el portal del paciente asociado.
RF11-RF12	Una medicación seleccionada o registrada queda visible en el portal del paciente asociado.
RF13-RF14	El portal muestra una visión agrupada de información vinculada al paciente autenticado.

6.12. Trazabilidad entre objetivos, requisitos y pruebas

Tabla 16. Trazabilidad funcional del análisis. Fuente: elaboración propia.

Objetivo específico	Requisito asociado	Prueba prevista
Registro por roles	RF01, RF02, RF03	Registro e inicio de sesión con médico y paciente.
Gestión básica de pacientes	RF04, RF05	Consulta de listado y edición de perfil.
Gestión de citas	RF06, RF07, RF08	Crear, consultar y cancelar cita con motivo.
Asignación de ejercicios	RF09, RF10	Seleccionar ejercicio del catálogo y verificar portal.
Asignación de medicación	RF11, RF12	Seleccionar medicamento y verificar portal.
Portal del paciente	RF13, RF14	Comprobar que el paciente visualiza información asociada.
Base relacional	RNF03	Comprobar tablas y relaciones con claves foráneas.

6.13. Priorización de requisitos

Para controlar el alcance se aplicó una priorización funcional. Los requisitos imprescindibles son aquellos sin los cuales la aplicación no podría demostrar su propósito principal. Los requisitos importantes completan el flujo, y los requisitos de mejora quedan documentados para futuras versiones. Esta priorización evita que el proyecto se disperse en funcionalidades secundarias antes de tener operativos los casos de uso principales.

Tabla 17. Priorización funcional del proyecto. Fuente: elaboración propia.

Prioridad	Requisitos incluidos	Justificación aplicada
Imprescindible	Registro, login, roles, pacientes y citas.	Sin estos elementos no existe flujo base de médico y paciente.
Importante	Asignación de ejercicios y medicación.	Aporta valor específico al caso de bruxismo y diferencia el proyecto de una agenda básica.
Importante	Portal del paciente.	Permite cerrar el ciclo de información y comprobar lo asignado por el médico.
Mejora	Recordatorios, estadísticas e informes.	Se consideran útiles, pero se excluyen para mantener alcance y seguridad.
Mejora	Historial clínico avanzado.	Requiere tratamiento de información más sensible y validación profesional.

6.14. Riesgos funcionales detectados en el análisis

El análisis también permitió identificar riesgos antes de la fase de diseño. El riesgo principal era confundir una aplicación de apoyo con una herramienta clínica. Para evitarlo, la memoria y la aplicación se centran en registrar información introducida por el usuario médico, sin emitir diagnósticos automáticos ni recomendaciones generadas por el sistema.

Otro riesgo era construir un modelo de datos demasiado amplio. Una base de datos con muchas tablas no utilizadas dificultaría la corrección y haría menos clara la defensa. Por ello se seleccionaron entidades directamente vinculadas a los objetivos: pacientes, médicos, citas, catálogos y asignaciones.

Tabla 18. Riesgos funcionales y medidas de control. Fuente: elaboración propia.

Riesgo	Impacto posible	Medida adoptada
Alcance clínico excesivo	El proyecto podría requerir validación sanitaria y legal no asumible en el TFG.	Limitar el sistema a gestión básica y declarar sus límites.
Módulos no conectados	La defensa sería menos clara y podrían aparecer errores no controlados.	Eliminar o no documentar elementos no integrados en el flujo real.
Datos inconsistentes	Citas o pautas podrían quedar sin paciente asociado.	Usar relaciones en base de datos y validaciones de backend.
Interfaz confusa por roles	El usuario podría acceder a funciones que no le corresponden.	Separar panel médico y panel paciente.

Con esta priorización y control de riesgos, el análisis queda conectado con el diseño posterior. Las decisiones técnicas no se toman de forma aislada, sino como respuesta a necesidades detectadas: rol de usuario, gestión de citas, pautas asignadas, integridad de datos y facilidad de prueba.

7. Diseño del proyecto

El diseño del proyecto convierte las necesidades detectadas en una solución técnica y visual. Este apartado explica la arquitectura, las decisiones de interfaz, la organización de carpetas, la comunicación entre capas y los elementos más importantes de la implementación sin incluir código fuente. La explicación se centra en cómo se ha construido el sistema y por qué se han elegido esas divisiones.

El diseño previo fue fundamental para evitar que la aplicación creciera sin control. Desde el inicio se decidió separar frontend, backend y base de datos. También se definieron vistas por rol y se estableció que el médico tendría acciones de gestión, mientras que el paciente tendría una experiencia de consulta y edición de sus propios datos.

7.1. Arquitectura de capas

La arquitectura se organiza en tres capas. La capa de presentación corresponde a React; la capa de servicios corresponde a FastAPI; la capa de persistencia corresponde a MySQL. Esta división evita que la interfaz conozca los detalles internos de la base de datos y permite que el backend concentre reglas de negocio y validación.

Tabla 15. Diseño por capas. Fuente: elaboración propia.

Capa	Responsabilidad	Elementos del proyecto
Presentación	Mostrar formularios, listados, paneles y mensajes según el rol.	Pantalla de login, registro, panel médico, portal paciente.
Servicios	Recibir peticiones, validar datos y coordinar operaciones.	Rutas de autenticación, pacientes, citas, ejercicios y medicación.
Persistencia	Guardar entidades y mantener relaciones.	Tablas de pacientes, médicos, citas, catálogos y asignaciones.

7.2. Diseño de interfaz y prototipos

El diseño de interfaz se planteó con pantallas simples y orientadas a tarea. La pantalla inicial permite iniciar sesión o registrarse. Una vez autenticado, el usuario accede a un panel distinto según su rol. Esta decisión reduce la carga visual y evita que el paciente vea opciones propias del médico.

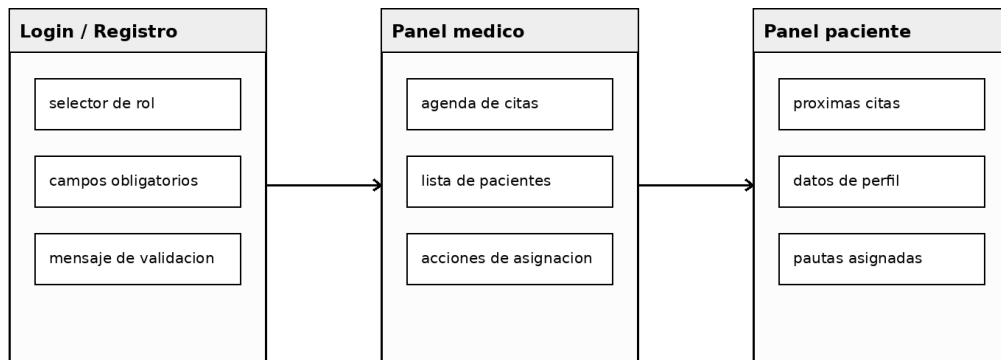


Figura 5. Wireframes principales de la aplicación. Fuente: elaboración propia.

Los mockups se han mantenido sencillos porque la prioridad del proyecto es la funcionalidad conectada. La aplicación necesita ser comprensible para la defensa, no competir en diseño visual con una plataforma comercial. Por ello se ha priorizado jerarquía clara, formularios directos y mensajes de error visibles.

7.3. Diseño del backend

El backend se organiza por módulos funcionales. Cada módulo agrupa operaciones relacionadas: autenticación, médicos, pacientes, citas, portal, ejercicios y medicación. Esta organización facilita localizar errores y permite explicar la aplicación por bloques durante la defensa.

El archivo principal inicializa la aplicación, configura la comunicación con el frontend y registra los módulos activos. Los modelos representan las tablas de la base de datos y los esquemas definen la forma de los datos que la API acepta o devuelve. Las operaciones de cada módulo reciben datos, validan condiciones básicas, consultan la base de datos y devuelven una respuesta estructurada.

El backend no se ha diseñado como una API pública de producción. Está pensado para ejecución local, pruebas académicas y demostración. Aun así, se han aplicado principios de separación: la lógica no está dentro del frontend y la base de datos no se manipula directamente desde el cliente.

Tabla 16. Organización funcional del backend. Fuente: elaboración propia.

Módulo backend	Funcionalidad	Motivo de separación
Autenticación	Registro e inicio de sesión por rol.	Centraliza el acceso inicial.
Pacientes	Consulta y actualización de datos de paciente.	Agrupar operaciones sobre perfil y listado.
Médicos	Consulta de profesionales disponibles.	Permite crear citas con profesional asociado.
Citas	Alta, consulta y cancelación de citas.	Gestiona agenda y estado de citas.

Módulo backend	Funcionalidad	Motivo de separación
Ejercicios	Catálogo y asignación de ejercicios.	Separa pautas físicas de otras entidades.
Medicamentos	Catálogo y asignación de medicación.	Separa tratamiento farmacológico de citas.
Portal paciente	Resumen de información asociada.	Agrupar información visible para el paciente.

7.4. Diseño del frontend

El frontend se ha diseñado como una aplicación React que cambia de vista según el estado de sesión y el rol del usuario. El estado de autenticación determina si se muestra la pantalla inicial, el panel médico o el panel paciente. Esta decisión permite que una única aplicación cliente atienda a los dos perfiles sin duplicar proyectos.

Las llamadas al backend se centralizan en un módulo de comunicación. De esta manera, los componentes de interfaz no necesitan conocer todos los detalles de las rutas. La centralización facilita modificar una dirección de servicio o ajustar el formato de una petición sin revisar toda la interfaz.

La interfaz del médico contiene acciones de gestión: consulta de pacientes, agenda, creación de citas y asignación de pautas. La interfaz del paciente contiene acciones de consulta: próximas citas, edición de perfil, ejercicios y medicaciones. Esta separación se corresponde con los casos de uso definidos en el análisis.

Tabla 17. Vistas principales del frontend. Fuente: elaboración propia.

Vista	Usuario	Elementos principales
Login	No autenticado	Formulario de acceso, enlace a registro y mensaje de error.
Registro	No autenticado	Selector de rol y campos obligatorios según perfil.
Panel médico	Médico	Agenda, listado de pacientes y acciones de asignación.
Panel paciente	Paciente	Citas, datos de perfil, ejercicios y medicaciones.
Gestión de citas	Médico y paciente	Listado, creación o cancelación según permisos.

7.5. Diseño de la base de datos

La base de datos se diseñó para responder a los requisitos funcionales sin añadir tablas innecesarias. El modelo permite registrar usuarios por rol, gestionar citas y asociar pautas a pacientes. Los catálogos separan la definición de ejercicios y medicamentos de las asignaciones concretas, lo que mejora la coherencia.

El diseño utiliza claves primarias para identificar registros y claves foráneas para mantener relaciones. Por ejemplo, una cita depende de un paciente; una asignación de ejercicio depende de un paciente y de un ejercicio del catálogo; y una medicación depende

de un paciente y puede referenciar un medicamento catalogado. Esto cumple el requisito de disponer de una base de datos relacional con varias tablas relacionadas.

Tabla 18. Diseño aplicado de tablas. Fuente: elaboración propia.

Tabla	Datos principales	Uso funcional
Pacientes	Identificación, contacto y credenciales de perfil.	Login de paciente, portal y relación con citas.
Medicos	Datos profesionales y credenciales de perfil.	Login de médico y asignación de agenda.
Citas	Fecha, estado, tipo y motivo de cancelación.	Gestión de agenda y consultas de próximas citas.
CatalogoEjercicios	Nombre, descripción e instrucciones.	Base para pautas asignables por médico.
EjerciciosPacientes	Paciente, ejercicio, frecuencia y fecha.	Pautas visibles en portal paciente.
CatalogoMedicamentos	Medicamento y descripción.	Base para medicación seleccionable.
Medicaciones	Paciente, dosis, frecuencia, motivo e inicio.	Medicación visible para el paciente.
TratamientoPacientes	Paciente, médico y tipo de seguimiento.	Preparada para ampliar seguimiento.

7.6. Diseño de servicios y comunicación

La comunicación entre frontend y backend se realiza mediante operaciones agrupadas por recurso. En la memoria no se reproduce código ni se listan fragmentos de implementación, pero sí se documenta qué tipo de operaciones existen y qué finalidad tienen.

Tabla 19. Diseño de servicios sin reproducción de código. Fuente: elaboración propia.

Grupo de servicios	Operaciones cubiertas	Interfaz que las utiliza
Autenticación	Registro y acceso por email y contraseña.	Login y registro.
Pacientes	Listado, consulta y actualización de datos.	Panel médico y panel paciente.
Citas	Crear, listar próximas citas y cancelar.	Agenda médica y citas del paciente.
Ejercicios	Consultar catálogo, asignar y retirar asignación.	Gestión de paciente y portal.
Medicamentos	Consultar catálogo, asignar y retirar asignación.	Gestión de paciente y portal.
Portal	Agrupar información relacionada con el paciente.	Panel paciente.

7.7. Decisiones de diseño y modificaciones realizadas

- Se decidió utilizar una API separada en lugar de mezclar lógica de datos dentro del frontend. Esta elección aumenta la claridad del proyecto y permite que la base de datos quede protegida por una capa intermedia.

- Se decidió usar catálogos para ejercicios y medicamentos, en lugar de permitir únicamente texto libre. Esta elección ayuda a mantener datos más consistentes y facilita mostrar opciones al médico durante la asignación.
- Se decidió no incorporar facturación, informes clínicos avanzados ni integración con correo electrónico. Estas funcionalidades son posibles, pero habrían ampliado el alcance y reducido la calidad de los módulos principales.
- Se decidió limpiar módulos no conectados antes de la entrega. Una memoria debe describir lo que el proyecto hace realmente, por lo que se priorizó coherencia frente a cantidad de archivos.

7.8. Seguridad prevista y límites del diseño

La seguridad del diseño actual es suficiente para una demostración académica local, pero no para producción. La aplicación trabaja con perfiles y contraseñas en un entorno de prueba, y la memoria reconoce que cualquier evolución real debe incluir cifrado de contraseñas, sesiones robustas, control de autorización por rol, HTTPS, auditoría de accesos y revisión legal.

Esta limitación no invalida el proyecto porque el objetivo académico es demostrar la construcción de una aplicación web completa. Sin embargo, sí condiciona las conclusiones y vías futuras. El sistema no debe utilizarse con datos reales de pacientes sin una transformación profunda en seguridad y cumplimiento normativo.

7.9. Diseño de navegación y experiencia de usuario

La navegación se ha diseñado para que el usuario siempre sepa en qué rol está trabajando. Tras el acceso, el médico entra en un panel orientado a gestión y el paciente en un panel orientado a consulta. Esta separación reduce errores y evita que un paciente encuentre opciones que no debe utilizar.

El diseño de experiencia de usuario se basa en formularios directos, listados visibles y mensajes de validación. Las acciones destructivas o relevantes, como cancelar una cita, exigen una causa. La aplicación no oculta los errores técnicos con mensajes genéricos, sino que intenta indicar al usuario qué dato falta o qué operación no puede realizarse.

Tabla 20. Decisiones de experiencia de usuario. Fuente: elaboración propia.

Decisión de interfaz	Aplicación concreta	Beneficio
Separación por rol	Panel médico y panel paciente diferenciados.	Reduce opciones innecesarias para cada usuario.
Formularios simples	Campos agrupados por operación.	Facilita pruebas y defensa del flujo.
Mensajes de validación	Avisos ante credenciales erróneas, duplicados o falta de motivo.	Ayuda a comprender reglas del sistema.
Listados de apoyo	Pacientes, citas, ejercicios y medicaciones visibles.	Permite comprobar resultados sin acceder a la base de datos.

7.10. Diseño de mantenimiento

El mantenimiento se ha tenido en cuenta separando responsabilidades. Los cambios visuales se realizan en el frontend; los cambios de reglas y validación se concentran en el backend; y los cambios de estructura persistente se reflejan en la base de datos. Esta división facilita que una mejora futura, como añadir recordatorios, pueda planificarse sin reescribir todo el sistema.

La documentación de módulos también forma parte del diseño de mantenimiento. Identificar qué módulo gestiona cada funcionalidad permite localizar con rapidez dónde se debería ampliar la aplicación: citas para agenda, ejercicios para pautas físicas, medicamentos para medicación y portal para la vista agrupada del paciente.

7.11. Correspondencia entre análisis y diseño

El diseño final mantiene relación directa con el análisis. Cada actor tiene una vista; cada requisito principal tiene un servicio; y cada entidad relevante se refleja en una tabla. Esta correspondencia evita que la memoria describa elementos no implementados y permite que la defensa se apoye en demostraciones concretas.

Tabla 21. Correspondencia entre análisis y diseño. Fuente: elaboración propia.

Elemento del análisis	Elemento de diseño	Evidencia en la aplicación
Actor médico	Panel médico	Gestión de pacientes, citas y pautas.
Actor paciente	Panel paciente	Consulta de citas, ejercicios y medicación.
Entidad cita	Tabla y servicios de citas	Alta, consulta y cancelación.
Entidad ejercicio	Catálogo y asignación	Selección por médico y consulta por paciente.
Entidad medicación	Catálogo y asignación	Registro por médico y consulta por paciente.
Requisito de integridad	Relaciones en MySQL	Asignaciones vinculadas a pacientes existentes.

8. Despliegue y pruebas

El despliegue del proyecto se ha planteado para un entorno local de corrección y defensa. La aplicación requiere ejecutar la base de datos, el backend y el frontend. Este apartado no reproduce código fuente, sino que describe el procedimiento, los requisitos previos y las pruebas funcionales realizadas para comprobar el comportamiento desde el punto de vista del usuario.

Las pruebas se han definido con enfoque de caja negra. Esto significa que se comprueba la funcionalidad observando entradas y salidas, sin basarse en la estructura interna del código. Cada prueba parte de un caso de uso y verifica si el sistema responde de forma esperada ante datos válidos o inválidos.

8.1. Requisitos previos de ejecución

Tabla 20. Requisitos de despliegue local. Fuente: elaboración propia.

Elemento	Uso durante el despliegue local	Comprobación esperada
Python 3	Ejecutar el backend desarrollado con FastAPI.	El entorno reconoce Python y permite instalar dependencias.
Node.js y npm	Ejecutar el frontend desarrollado con React y Vite.	El gestor de paquetes instala dependencias del cliente.
MySQL	Crear la base de datos relacional del proyecto.	La base de datos acepta scripts y mantiene tablas.
Dependencias backend	Librerías necesarias para servidor, validación y acceso a datos.	La API arranca sin errores de importación.
Dependencias frontend	Librerías necesarias para construir la interfaz.	La aplicación cliente se ejecuta y compila.
Navegador web	Acceso a la interfaz de usuario.	El usuario puede abrir la aplicación local.

8.2. Preparación de la base de datos

La preparación de la base de datos consiste en crear el esquema relacional del proyecto y cargar los catálogos necesarios para las pruebas. Primero se crea la base de datos del sistema; después se ejecutan los scripts de creación de tablas; finalmente se cargan datos de catálogo para ejercicios y medicamentos. Esta secuencia permite que las pruebas de asignación dispongan de opciones seleccionables.

La comprobación del despliegue de base de datos se realiza revisando que existan las tablas principales y que las relaciones no impidan insertar datos válidos. También se comprueba que los catálogos contengan registros antes de probar la asignación de ejercicios o medicación.

8.3. Despliegue del backend

El backend se despliega en local desde la carpeta correspondiente del proyecto. Antes de arrancarlo se instalan las dependencias de Python indicadas en el archivo de requisitos. Una

vez iniciado el servidor, la API queda disponible para recibir peticiones del frontend y para consultar la documentación automática de servicios. En la corrección, esta documentación permite verificar de forma visual qué operaciones expone el sistema.

La comprobación principal del backend consiste en confirmar que la aplicación arranca sin errores, que conecta con la base de datos y que las rutas principales responden. Si la base de datos no está creada o las credenciales locales no coinciden, la API no podrá completar las operaciones de persistencia.

8.4. Despliegue del frontend

El frontend se ejecuta desde su carpeta de cliente. Primero se instalan las dependencias del proyecto y después se inicia el servidor de desarrollo. El navegador carga la aplicación y el usuario puede acceder a las pantallas de login, registro y paneles. Para la entrega también se comprueba que el cliente pueda generar una versión compilada sin errores.

La integración se valida realizando acciones desde la interfaz y comprobando que el backend responde. El flujo mínimo de integración consiste en registrar usuarios, iniciar sesión, crear una cita, asignar un ejercicio y consultar la información desde el portal del paciente.

8.5. Estrategia de pruebas de caja negra

Las pruebas se han diseñado a partir de los casos de uso. Cada prueba define una entrada o acción del usuario, un resultado esperado y un resultado obtenido. Se incluyen pruebas positivas y negativas para comprobar tanto el funcionamiento correcto como la respuesta ante errores frecuentes.

Tabla 21. Pruebas funcionales de caja negra. Fuente: elaboración propia.

ID	Prueba	Entrada / acción	Resultado esperado	Resultado
P01	Registro de paciente válido	Datos obligatorios completos y correo no existente.	El sistema crea el paciente y muestra confirmación.	Correcto
P02	Registro con correo duplicado	Correo ya usado en el sistema.	El sistema rechaza el registro y muestra error.	Correcto
P03	Registro de médico válido	Datos personales y profesionales completos.	El sistema crea el médico.	Correcto
P04	Inicio de sesión correcto	Email y contraseña existentes.	El sistema carga el panel según rol.	Correcto
P05	Inicio de sesión incorrecto	Credenciales no válidas.	El sistema muestra mensaje de error.	Correcto
P06	Consulta de pacientes	Médico autenticado.	Se muestra listado de pacientes.	Correcto
P07	Edición de perfil	Paciente modifica datos permitidos.	Los cambios quedan	Correcto

ID	Prueba	Entrada / acción	Resultado esperado	Resultado
			guardados.	
P08	Creación de cita válida	Paciente, médico y fecha futura válidos.	La cita queda registrada.	Correcto
P09	Cita duplicada	Mismo médico y misma fecha ya ocupada.	El sistema evita duplicidad básica.	Correcto
P10	Cancelación sin motivo	Usuario intenta cancelar sin causa.	El sistema solicita motivo obligatorio.	Correcto
P11	Cancelación con motivo	Usuario indica causa.	La cita pasa a estado cancelado.	Correcto
P12	Asignación de ejercicio	Médico selecciona paciente y ejercicio.	El paciente ve el ejercicio en su portal.	Correcto
P13	Asignación de medicación	Médico registra medicamento, dosis y frecuencia.	El paciente ve la medicación asignada.	Correcto
P14	Portal paciente	Paciente autenticado con datos asociados.	Se muestran citas y pautas vinculadas.	Correcto

8.6. Resultados de las pruebas

Las pruebas principales finalizaron correctamente en el entorno local. El sistema permitió completar el flujo de registro, login, gestión de citas y asignación de pautas. Las pruebas negativas también permitieron comprobar que la aplicación no acepta determinadas acciones incompletas, como cancelar una cita sin indicar motivo.

La integración entre frontend y backend fue la parte que requirió más ajustes. Algunas respuestas del backend tenían que adaptarse para que la interfaz mostrara mensajes útiles. También fue necesario revisar campos obligatorios para que los formularios enviaran información compatible con los esquemas de validación.

La construcción del frontend se utilizó como comprobación adicional. Si el cliente genera una versión final sin errores, se reduce el riesgo de entregar un proyecto que solo funciona en modo desarrollo. La importación del backend y el arranque de la API verifican que la estructura de paquetes y dependencias es correcta.

8.7. Incidencias y limitaciones detectadas

- La autenticación es básica y no representa una solución profesional para datos reales.
- No se han incorporado pruebas automatizadas, por lo que la validación depende de pruebas manuales.
- El despliegue se ha realizado en local y no en un servidor remoto.
- La interfaz cumple la función académica, pero podría mejorar en diseño responsive y accesibilidad avanzada.
- El sistema no incluye recordatorios automáticos ni comunicación por correo electrónico.

- El historial clínico avanzado queda fuera del alcance para evitar tratar información sanitaria sensible sin garantías suficientes.

Estas limitaciones se consideran aceptables para el alcance del TFG siempre que queden documentadas. Además, sirven como base para las vías futuras, ya que señalan qué mejoras serían prioritarias si el proyecto evolucionara.

9. Conclusiones

El proyecto ha alcanzado el objetivo general propuesto: desarrollar una aplicación web funcional para la gestión básica de pacientes con bruxismo. Se ha construido una solución con frontend, backend y base de datos relacional, y se han conectado los módulos principales mediante una API. La aplicación permite trabajar con dos perfiles de usuario, médico y paciente, y ofrece funcionalidades adaptadas a cada uno.

Entre los objetivos específicos alcanzados se encuentran el registro por roles, el inicio de sesión, la consulta de pacientes, la edición del perfil del paciente, la gestión de citas, la cancelación con motivo, la asignación de ejercicios, la asignación de medicación y la consulta de información desde el portal del paciente. Estos objetivos son funcionales y verificables, por lo que pueden demostrarse durante la defensa.

También se ha cumplido el requisito de utilizar una base de datos relacional con más de tres tablas relacionadas. El modelo incluye pacientes, médicos, citas, catálogos y asignaciones, conectados mediante relaciones que evitan que la información quede aislada. Esta parte ha sido especialmente importante porque permite explicar el sistema como una aplicación completa y no como una colección de pantallas.

Desde el punto de vista técnico, el proyecto ha permitido aplicar conocimientos de desarrollo web en un escenario integrado. El frontend con React se comunica con un backend FastAPI, y el backend utiliza SQLAlchemy para persistir datos en MySQL. Esta separación por capas ha facilitado la organización del código y la explicación del funcionamiento general.

La principal dificultad ha sido la integración entre partes. Crear servicios o pantallas por separado resulta más sencillo que hacer que todas las operaciones funcionen de extremo a extremo. La gestión de campos obligatorios, la coherencia de respuestas y la sincronización entre la base de datos y la interfaz han requerido revisiones sucesivas.

Otra dificultad relevante ha sido delimitar el alcance. Al tratarse de un tema sanitario, resulta tentador añadir historial clínico, informes, recordatorios o métricas avanzadas. Sin embargo, esas funciones habrían aumentado la complejidad y habrían introducido riesgos relacionados con datos sensibles. La decisión final ha sido mantener el sistema en un nivel de gestión básica y documentar sus límites.

La experiencia de desarrollo ha sido positiva porque ha obligado a tomar decisiones similares a las de un proyecto real: elegir tecnologías, justificar herramientas, modelar datos, probar flujos y preparar documentación. También ha permitido comprobar que la limpieza final del proyecto es tan importante como la implementación, ya que una entrega con módulos incoherentes dificulta la corrección y la defensa.

En conclusión, el resultado es un proyecto académico funcional, coherente y ampliable. No está preparado para producción ni para uso con datos reales de pacientes, pero cumple su objetivo formativo y demuestra competencias de análisis, diseño, implementación, despliegue local y pruebas funcionales en una aplicación web completa.

El balance final de objetivos es positivo. Las funcionalidades críticas se han implementado y documentado, mientras que las mejoras no incorporadas se han trasladado de forma razonada al apartado de vías futuras. Esta separación evita presentar como terminado aquello que no corresponde a la versión actual y refuerza la honestidad técnica de la memoria.

La realización del proyecto también ha demostrado la importancia de la planificación. Las fases iniciales permitieron avanzar con orden, pero la integración final mostró que los sistemas web requieren pruebas repetidas y ajustes pequeños. Esta experiencia resulta especialmente útil para un entorno laboral, donde la entrega no depende solo de programar, sino de integrar, documentar y validar.

10. Vías futuras

Las vías futuras recogen mejoras que no se han incluido en la versión actual por limitación de alcance, tiempo o seguridad. Algunas de estas mejoras ya estaban presentes como ideas iniciales, pero se descartaron para mantener una entrega académica controlada. Otras aparecen como consecuencia de las pruebas y de la reflexión final.

La primera vía futura es reforzar la seguridad. El sistema debería almacenar contraseñas de forma cifrada, utilizar un sistema de autenticación robusto, gestionar expiración de sesiones y aplicar autorización real por rol. Además, si se manejaran datos reales de pacientes, sería imprescindible incorporar HTTPS, auditoría, copias de seguridad y cumplimiento de normativa de protección de datos.

La segunda vía futura es ampliar el historial del paciente. La aplicación actual permite consultar citas, ejercicios y medicación, pero no incluye un historial clínico detallado. Una evolución podría registrar síntomas, evolución del dolor, observaciones del profesional, revisiones y documentos asociados. Esta mejora tendría que diseñarse con especial prudencia porque aumentaría la sensibilidad de los datos tratados.

La tercera mejora sería incorporar recordatorios. El sistema podría enviar avisos de próximas citas o recordatorios de ejercicios asignados. Esta funcionalidad aportaría valor al paciente, pero requeriría configurar comunicación externa, consentimiento, plantillas de mensajes y protección frente a exposición de datos sensibles.

Otra vía futura es añadir pruebas automatizadas. La versión actual se ha validado mediante pruebas manuales de caja negra. Para mejorar calidad, se podrían incorporar pruebas unitarias del backend, pruebas de integración sobre la API y pruebas de interfaz. Esto permitiría detectar regresiones cuando se añadan nuevas funcionalidades.

El despliegue remoto también queda como mejora. La aplicación se ha ejecutado en local, pero una versión evolucionada podría desplegarse en un servidor o plataforma cloud. Este paso exigiría variables de entorno, configuración de seguridad, base de datos remota, dominio, HTTPS y control de copias de seguridad.

A nivel de interfaz, el frontend podría incorporar diseño responsive avanzado, componentes reutilizables, rutas internas más claras y mejoras de accesibilidad. También sería posible añadir paneles con indicadores básicos, como número de citas próximas, ejercicios activos o medicaciones vigentes. Estas mejoras deberían mantenerse dentro de un enfoque de apoyo organizativo.

Finalmente, una evolución profesional requeriría validación con usuarios reales. Antes de implantar la herramienta en una consulta, sería necesario observar cómo trabaja el profesional, qué información necesita realmente el paciente y qué procesos no deberían

digitalizarse sin supervisión. Esta validación permitiría transformar el prototipo académico en un producto más útil y responsable.

También podría mejorarse la gestión documental. En una versión futura se podrían adjuntar documentos informativos, consentimientos o recomendaciones generales revisadas por el profesional. Esta mejora tendría que diferenciar claramente entre documentos administrativos y documentación sanitaria, aplicando permisos y conservación adecuados.

Otra evolución posible es incorporar un sistema de roles más detallado. La versión actual distingue médico y paciente, pero una consulta real podría necesitar administradores, recepcionistas o profesionales con distintos permisos. Esta mejora permitiría acercar el prototipo a un contexto laboral sin convertirlo todavía en una plataforma clínica completa.

Por último, el proyecto podría incorporar métricas de calidad de uso. Sin tratar datos clínicos sensibles, sería posible medir operaciones generales como número de citas creadas, citas canceladas o pautas activas. Esta información ayudaría a evaluar la utilidad de la herramienta y a orientar mejoras de interfaz.

11. Bibliografía/Webgrafía

La bibliografía se presenta siguiendo un formato inspirado en APA. Las fuentes se han utilizado para contextualizar el bruxismo, la salud digital, la protección de datos sanitarios y la documentación técnica de las tecnologías empleadas. Fecha de consulta: 6 de junio de 2026.

Agencia Española de Protección de Datos. (s. f.). Guía para profesionales del sector sanitario. <https://www.aepd.es/guias/guia-profesionales-sector-sanitario.pdf>

FastAPI. (s. f.). FastAPI documentation. <https://fastapi.tiangolo.com/>

Mayo Clinic. (s. f.). Teeth grinding (bruxism): Symptoms and causes. <https://www.mayoclinic.org/diseases-conditions/bruxism/symptoms-causes/syc-20356095>

Mozilla Developer Network. (s. f.). JavaScript documentation. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

MySQL. (s. f.). MySQL documentation. <https://dev.mysql.com/doc/>

National Health Service. (2026). Teeth grinding (bruxism). <https://www.nhs.uk/symptoms/teeth-grinding/>

Pydantic. (s. f.). Pydantic documentation. <https://pydantic.dev/docs/>

Python Software Foundation. (s. f.). Python documentation. <https://www.python.org/doc/>

React. (s. f.). React documentation. <https://react.dev/>

SQLAlchemy. (s. f.). SQLAlchemy documentation. <https://docs.sqlalchemy.org/>

Vite. (s. f.). Vite documentation. <https://vite.dev/guide/>

World Health Organization. (2021). Global strategy on digital health 2020-2025. <https://www.who.int/publications/i/item/9789240020924>

12. Anexos

Los anexos incluyen información operativa que resulta útil para ejecutar y usar el proyecto, pero que no debe interrumpir el cuerpo principal de la memoria. Se incluyen dos anexos obligatorios: manual de instalación y manual de usuario.

12.1. Manual de instalación

12.1.1. Objetivo del manual

Este manual describe el procedimiento para preparar la aplicación en un entorno local de revisión. Se asume que el evaluador dispone del proyecto completo, de un equipo con permisos de instalación y de las herramientas indicadas en el apartado de despliegue.

12.1.2. Preparación del entorno

1. Instalar Python 3 y comprobar que el sistema lo reconoce.
2. Instalar Node.js y npm para ejecutar la aplicación frontend.
3. Instalar MySQL o disponer de un servidor MySQL local accesible.
4. Abrir el proyecto en un editor de código o entorno de desarrollo.
5. Revisar la configuración de conexión a base de datos antes de arrancar el backend.

12.1.3. Creación de la base de datos

La base de datos se prepara ejecutando primero el script de creación del esquema y después el script de carga de catálogos. El primer script crea las tablas y relaciones necesarias; el segundo añade ejercicios y medicamentos de ejemplo para poder realizar pruebas de asignación.

Tabla 22. Instalación de base de datos. Fuente: elaboración propia.

Paso	Acción	Resultado esperado
1	Crear la base de datos del proyecto en MySQL.	Existe un esquema vacío preparado para recibir tablas.
2	Ejecutar el script de creación incluido en la carpeta de base de datos.	Se crean tablas, claves primarias y relaciones.
3	Ejecutar el script de catálogos.	Existen ejercicios y medicamentos para pruebas.
4	Revisar tablas desde un cliente MySQL.	Se confirma que el esquema está completo.

12.1.4. Arranque del backend

Para arrancar el backend se accede a la carpeta del servidor, se instalan las dependencias del proyecto y se inicia la aplicación con el servidor ASGI configurado. La comprobación correcta es que el servidor quede escuchando en el puerto local definido y que la documentación de servicios sea accesible desde el navegador.

12.1.5. Arranque del frontend

Para arrancar el frontend se accede a la carpeta cliente, se instalan las dependencias de Node.js y se inicia el servidor de desarrollo. La aplicación debe abrirse en el navegador y mostrar la pantalla inicial de acceso y registro. Si el backend no está activo, los formularios cargarán, pero las operaciones que dependan de la API no se completarán.

12.1.6. Comprobación final de instalación

6. Registrar un usuario médico de prueba.
7. Registrar un usuario paciente de prueba.
8. Iniciar sesión con el médico y comprobar que aparece su panel.
9. Crear una cita para el paciente registrado.
10. Asignar un ejercicio o medicación desde el panel médico.
11. Iniciar sesión con el paciente y comprobar que la información aparece en su portal.

12.2. Manual de usuario

12.2.1. Acceso a la aplicación

El usuario accede a la aplicación mediante el navegador una vez que frontend y backend están activos. La pantalla inicial permite iniciar sesión o acceder al registro. La aplicación diferencia entre médico y paciente, por lo que el tipo de usuario seleccionado durante el registro condiciona el panel que se mostrará después.

12.2.2. Registro de usuario

12. Entrar en la pantalla de registro desde la página inicial.
13. Seleccionar el tipo de usuario: paciente o médico.
14. Completar los campos obligatorios que se muestran para el rol seleccionado.
15. Enviar el formulario y revisar el mensaje de confirmación o error.
16. Si el correo ya existe, usar otro correo o iniciar sesión con la cuenta existente.

12.2.3. Inicio de sesión

17. Introducir email y contraseña.
18. Pulsar la opción de acceso.
19. Esperar a que la aplicación cargue el panel correspondiente.
20. Si las credenciales no son correctas, revisar el mensaje de error y repetir la operación.

12.2.4. Uso del panel médico

El panel médico agrupa las acciones de gestión. El médico puede consultar pacientes, revisar agenda, crear citas y asignar pautas. Las acciones se orientan a pacientes ya registrados, por lo que es recomendable crear primero al menos un paciente de prueba.

Tabla 23. Acciones del panel médico. Fuente: elaboración propia.

Acción del médico	Pasos principales	Resultado
Consultar pacientes	Acceder a la sección de pacientes.	Se muestra el listado disponible.
Crear cita	Seleccionar paciente, profesional, fecha, tipo y guardar.	La cita queda registrada si no hay conflicto.
Cancelar cita	Seleccionar cita, introducir motivo y confirmar.	La cita pasa a estado cancelado.
Asignar ejercicio	Elegir paciente, ejercicio del catálogo y frecuencia.	El ejercicio aparece en el portal del paciente.
Asignar medicación	Elegir paciente, medicamento, dosis, frecuencia y motivo.	La medicación aparece asociada al paciente.

12.2.5. Uso del panel paciente

El panel del paciente muestra información relacionada con su perfil. El paciente puede consultar próximas citas, revisar ejercicios y medicaciones asignadas y modificar datos permitidos de su perfil. El objetivo es que tenga una vista centralizada de sus pautas sin acceder a funciones de gestión profesional.

Tabla 24. Acciones del panel paciente. Fuente: elaboración propia.

Acción del paciente	Pasos principales	Resultado
Consultar citas	Entrar en la sección de citas del panel.	Se muestran próximas citas asociadas.
Cancelar cita	Seleccionar cita, indicar motivo y confirmar.	La cita queda cancelada.
Editar perfil	Abrir datos personales, modificar campos y guardar.	Los datos quedan actualizados.
Ver ejercicios	Abrir sección de ejercicios.	Se muestran pautas asignadas por el médico.
Ver medicaciones	Abrir sección de medicación.	Se muestran medicaciones asociadas.

12.2.6. Errores frecuentes

- Correo ya registrado: el sistema no permite duplicar usuarios con el mismo email.
- Credenciales incorrectas: el email o la contraseña no coinciden con ningún usuario registrado.
- Paciente no encontrado: se intenta operar sobre un paciente que no existe o no está seleccionado.
- Cita duplicada: ya existe una cita para el mismo profesional en la fecha indicada.
- Motivo de cancelación obligatorio: no se puede cancelar una cita sin indicar causa.
- Catálogo vacío: antes de asignar ejercicios o medicación deben cargarse los datos iniciales.

12.2.7. Recomendaciones de uso

- Usar datos de prueba durante la defensa y no datos reales de pacientes.
- Crear primero un médico y un paciente para poder probar los flujos principales.
- Usar fechas futuras para comprobar correctamente la sección de próximas citas.
- Revisar mensajes de error porque explican la validación que no se ha cumplido.
- Mantener activo el backend antes de utilizar formularios que consultan o guardan datos.